Scaling Spectrally Ordered Neighborhood Training (SONT): Multi-Pole Spherical Routing, Deep Surprisal Backpropagation, and Buffer-Free Continual Learning in PyTorch

Toiab Zahoor
toiabzahoor@gmail.com

Abstract

In my earlier paper on Spectrally Ordered Neighborhood Training (SONT) [1], the core idea was simple. We lock every weight block to the unit sphere $\mathcal{S}^{d-1}$ ($\|W_i\|_2 = 1$) and keep the blocks sorted by their angle $\theta(W_i) = \arccos(\langle W_i, u_0 \rangle)$ from a fixed pole u_0. Because the blocks stay sorted, routing is no longer a slow $\mathcal{O}(N \cdot d)$ matrix multiplication. Instead, it turns into an exact $\mathcal{O}(\log_2 N)$ binary search over a single error valley. Once binary search finds the best block i^*, we only update a local window $[i^* - r, i^* + r]$ whose radius r comes from the Shannon surprisal $\mathcal{I}(x, y; W_{i^*}) = -\log_2 P(y \mid x; W_{i^*})$. When a pattern is already familiar, r drops to 0 and training stops on its own. When a new feature shows up at that same angle, r grows and wakes up neighbor blocks next to i^* so the old block does not get erased.

That setup worked well for a single layer on small data. However, when we try to scale SONT to deep networks and large vision datasets in PyTorch, four hard problems show up right away. In this paper, I work through all four of them. First, if we use linear PCA on raw pixels, the feature space drifts between tasks. I prove that using a frozen feature backbone together with a maturity gate $\psi(M_j) = (1 - M_j)^\beta$ locks finished blocks so their weights do not drift at all ($\|\Delta W_{j^*}\|_F = 0$). Second, in high dimensions ($d \in [512, 4096]$), almost all vectors on a sphere pile up right on the equator ($\theta \approx \pi/2$), which ruins single-pole sorting. I fix this by building a Multi-Pole Spherical k-D Tree across k orthogonal poles, which spreads the blocks back out and still routes in $\mathcal{O}(k \log_2 N)$ steps. Third, hidden layers inside a deep network do not have ground-truth labels y during the forward pass. I split the routing into two parts: the forward pass routes purely by input angle, while the backward pass uses the norm of the backpropagated gradient to set the surprisal radius $r^{(\ell)}$. Fourth, because SONT updates a contiguous slice of memory $[i^* - r, i^* + r]$ instead of random scattered experts, a GPU can read the whole neighborhood in one burst without running a router matrix multiply.

I tested all of this in PyTorch on an NVIDIA Tesla T4 GPU. On 5-Task Split Fashion-MNIST, SONT takes just 223.6 M FLOPs (270× to 400× less compute than Dense, EWC, and Switch Top-8 MoE) and keeps 42.00% Task 1 accuracy with zero buffer, while the buffer-free baselines drop to 0.00%. On a 10-task, 200-class Split ImageNet-R benchmark using native 224×224 ViT-B/16 features, single-pole SONT gets crowded at the equator and only holds 7.35% on Task 1. Switching to 4-Pole SONT pushes Task 1 accuracy up to 32.01% and reaches 20.20% across all 200 classes with zero replay buffer. That beats AdamW (0.00%), EWC (0.00%), Switch MoE (0.61%), and even a 1,000-image Experience Replay buffer (5.05% Task 1, 14.88% average) while using 14× fewer FLOPs. On a 4-layer deep SONT network trained on 200-class ImageNet-R, test accuracy reaches 60.67%, and the hidden-layer backward radius shrinks from 1.06 in Epoch 1 down to 0.01 in Epoch 8, turning off 99.5% of hidden updates on its own. Finally, at $N = 4096, d = 1024$, contiguous-slice SONT takes 0.2083 ms per batch on a Tesla T4—making it 7.19× faster than Dense (1.4982 ms) and 8.32× faster than Switch Top-9 MoE (1.7338 ms).

Keywords: Spectrally Ordered Neighborhood Training (SONT), Routerless Mixture of Experts, Unit-Sphere Spectral Ordering, Shannon Surprisal, Buffer-Free Continual Learning, Spherical k-D Trees.

1 Introduction

In a normal dense neural network, every weight block is updated on every batch of data. It is simple to code, but it costs $\mathcal{O}(N \cdot d)$ operations per layer. Worse, when the data stream changes from one task to another, the new gradients overwrite the old weights. Before long, the network forgets the earlier tasks completely [7]. Sparse Mixture-of-Experts (MoE) layers [3,4] try to save compute by only using $k \ll N$ experts per token. But to figure out which k experts to use, MoE still has to multiply the input by a learned router matrix of size $N \times d$. That brings the $\mathcal{O}(N \cdot d)$ cost right back. On top of that, learned routers need extra load-balancing losses so they do not dump every token into the same two experts, and they pull expert weights from random places in GPU memory [1].

In the first SONT paper [1], I dropped the learned router completely. Instead, we put two geometric rules on a layer of N weight blocks $\mathcal{W} = \{W_1, \ldots, W_N\}$:

1. **Unit-Sphere Normalization and Angle Sorting:** Every block is normalized to sit on the unit sphere $\mathcal{S}^{d-1}$ ($\|W_i\|_2 = 1$). We pick a fixed unit pole $u_0 \in \mathcal{S}^{d-1}$ and keep the blocks sorted by their polar angle $\theta(W_i) = \arccos(\langle W_i, u_0 \rangle) \in [0, \pi]$:

$$\theta(W_1) \leq \theta(W_2) \leq \cdots \leq \theta(W_N). \tag{1}$$

   By Theorem 3.2 of [1], sorting the angles makes the alignment error $E(i; x, y) = \mathcal{L}_0 - \alpha \cos(\theta(W_i) - \phi(x, y))$ form one clean valley over the indices $\{1, \ldots, N\}$. We do not need to check all N blocks. A basic binary search finds the best block $i^* = \arg\min_i E(i; x, y)$ in at most $\lceil \log_2 N \rceil + 1$ steps.

2. **Shannon Surprisal Window:** Once we have i^*, we only update blocks inside a window $\mathcal{N}(x, y) = [i^* - r(x, y), i^* + r(x, y)]$. The radius $r(x, y)$ comes from the sample's Shannon surprisal in bits [1,5]:

$$\mathcal{I}(x, y; W_{i^*}) = -\log_2 P(y \mid x; W_{i^*}) = \frac{1}{\ln 2} E(i^*; x, y), \quad r(x, y) = \min\left(R_{\max}, \left\lfloor \frac{\mathcal{I}(x, y; W_{i^*})}{2\kappa} \right\rfloor\right). \tag{2}$$

   Theorem 4.1 of [1] shows why this is useful. When block i^* already knows the pattern well, $P(y \mid x; W_{i^*}) \to 1$. The surprisal $\mathcal{I}$ falls below 2κ, the radius r becomes 0, and backpropagation stops by itself. But if a sample lands at that same polar angle ϕ with a new feature in the remaining $d - 1$ dimensions, the error is high ($\mathcal{I} \gg 2\kappa$). Then r opens up and recruits fresh neighbor blocks in $[i^* - r, i^* + r] \setminus \{i^*\}$ rather than overwriting W_{i^*}.

Where the First Version Runs Into Trouble. On a single layer with $N = 1024$ and $d = 64$ (65,536 parameters), SONT matched a full $\mathcal{O}(N)$ scan 100% of the time and cut FLOPs by 88.57× [1]. Still, that was a small setup. If we want SONT to work on real deep networks and large datasets, four things go wrong:

1. **Linear PCA Drift:** In my first Fashion-MNIST tests, Task 1 accuracy stayed at 95.8% right after Task 2, but slid down to 42.0% by the end of Task 5. That happens because linear PCA on raw pixels does not separate all 10 classes cleanly, and old blocks still get nudged slightly when new tasks arrive nearby.

2. **Everything Crowds at the Equator in High Dimensions:** When $d = 64$, polar angles spread out fine. When we go up to $d = 512$ or $d = 4096$, high-dimensional spheres behave strangely. Almost all random unit vectors sit right at $\theta \approx \pi/2$ (the equator), with variance $1/d$. If all N blocks crowd into the same tiny band around $\pi/2$, sorting by one pole stops helping.

3. **Hidden Layers Have No Labels y:** In a 1-layer classifier, we know the true label y right away. If we stack 4 or 12 SONT layers deep, Layer 1 and Layer 2 do not have a label y during the forward pass. We need a way to route forward without y, and a way to measure surprisal on the way back.

4. **Python Loops Are Slow on GPUs:** In Table 3 of [1], SONT beat Top-8 MoE by 30.22× in FLOPs, yet took 2.461 s in Python compared to 0.960 s for MoE. Running a 'for' loop over batch samples on the CPU wastes what the GPU is actually good at.

How This Paper Fixes It. Here, I address each of those four points directly:

- **Spherical Fiber Split and Maturity Locking (Sections 2 & 3.1):** I write out the exact dot-product split between the polar angle and the $(d-1)$-dimensional fiber sphere (Lemma 2.1). Then in Theorem 3.1, I show that pairing a frozen feature extractor with a maturity gate $\psi(M_j) = (1-M_j)^\beta$ keeps finished blocks at strictly zero drift ($\|\Delta W_{j^*}\|_F = 0$) across all future tasks.

- **Multi-Pole Spherical k-D Trees (Section 3.2):** Building on Remark 3.4 of [1], I use k mutually orthogonal poles $U = [u_1, \ldots, u_k] \in \mathrm{St}(d, k)$ to split the sphere into sectors before sorting by angle (Theorem 3.3). This fixes the equator crowding problem at $d = 512$ and $d = 4096$ while keeping routing at $\mathcal{O}(k \log_2 N)$ steps.

- **Decoupled Deep Routing and Contiguous GPU Slicing (Sections 3.3 & 3.4):** Theorem 3.4 decouples the forward pass (which routes only by input angle) from the backward pass (which uses the backpropagated gradient norm $\|\delta^{(\ell)}\|_2$ to set the window radius $r^{(\ell)}$). Proposition 3.5 shows why reading SONT's contiguous block slice $[i^* - r, i^* + r]$ in one burst beats the router matrix multiply and scattered gathers of Top-K MoE on real GPU hardware.

- **Benchmarks Up to 200-Class ImageNet-R and ImageNet-A (Section 4):** Alongside the compact Fashion-MNIST and ECG5000 tests, I run full PyTorch benchmarks on an NVIDIA Tesla T4 using native 224×224 ViT-B/16 embeddings across 30,000 images of 200-Class ImageNet-R and 7,500 adversarial images of ImageNet-A.

2 PyTorch Architecture: Polar Routing and Orthogonal Fibers

Suppose we are given a stream of inputs $x \in \mathbb{R}^D$ and labels $y \in \{1, \ldots, C\}$.

2.1 Stiefel Projection and Fiber Decomposition

Let $P \in \mathrm{St}(D, d)$ ($P^T P = I_d$, $d \leq D$) be a fixed orthonormal matrix found via SVD and QR completion on centered initial features $X - \mu$ [2]. We map each input $x \in \mathbb{R}^D$ onto the unit sphere $\mathcal{S}^{d-1}$:

$$z = (x - \mu)P \in \mathbb{R}^d, \quad \hat{z} = \frac{z}{\|z\|_2} \in \mathcal{S}^{d-1}, \quad \phi(x) = \arccos(\langle \hat{z}, u_0 \rangle) \in [0, \pi]. \tag{3}$$

Just like in Section 6 of [1], each expert block $i \in \{1, \ldots, N\}$ has a unit routing key $w_i \in \mathcal{S}^{d-1}$. We split w_i into a component along the pole $u_0 = e_1 \in \mathcal{S}^{d-1}$ and an orthogonal fiber vector $f_i \in \mathcal{S}^{d-2}$:

$$w_i = \cos(\theta_i)u_0 + \sin(\theta_i)f_i, \quad \|f_i\|_2 = 1, \quad \langle f_i, u_0 \rangle = 0, \tag{4}$$

where $0 \leq \theta_1 \leq \theta_2 \leq \cdots \leq \theta_N \leq \pi$. Each block also holds its local linear classifier weights $(W_i^{\text{exp}} \in \mathbb{R}^{d \times C}, b_i^{\text{exp}} \in \mathbb{R}^C)$ and a maturity number $M_i \in [0, 1]$ that tracks how consolidated that block is.

Lemma 2.1 (Exact Spherical Decomposition of Alignment Error)**.** *Take any unit input* $\hat{z} = \cos(\phi)u_0 + \sin(\phi)f_z \in \mathcal{S}^{d-1}$ *and unit routing key* $w_i = \cos(\theta_i)u_0 + \sin(\theta_i)f_i \in \mathcal{S}^{d-1}$*, where* $\|u_0\|_2 = \|f_z\|_2 = \|f_i\|_2 = 1$ *and* $\langle u_0, f_z \rangle = \langle u_0, f_i \rangle = 0$*. Their inner product on* $\mathcal{S}^{d-1}$ *is:*

$$\langle w_i, \hat{z} \rangle = \cos(\theta_i - \phi) - \sin(\theta_i)\sin(\phi)\big(1 - \langle f_i, f_z \rangle\big). \tag{5}$$

Because of this, finding the largest dot product $\langle w_i, \hat{z} \rangle$ *splits into two steps: (i) getting the closest 1D polar angle* $|\theta_i - \phi|$ *in* $\mathcal{O}(\log_2 N)$ *steps with binary search, and (ii) checking the orthogonal fiber match* $\langle f_j, f_z \rangle$ *inside the small local slice* $j \in [i^* - R_{\max}, i^* + R_{\max}]$*.*

Proof. Let us expand the dot product $\langle w_i, \hat{z} \rangle$. Since u_0 is perpendicular to both f_i and f_z, the cross terms $\langle u_0, f_z \rangle$ and $\langle f_i, u_0 \rangle$ are zero, and $\langle u_0, u_0 \rangle = 1$. That leaves:

$$\langle w_i, \hat{z} \rangle = \cos(\theta_i)\cos(\phi) + \sin(\theta_i)\sin(\phi)\langle f_i, f_z \rangle.$$

From basic trigonometry, $\cos(\theta_i - \phi) = \cos(\theta_i)\cos(\phi) + \sin(\theta_i)\sin(\phi)$, which means we can replace $\cos(\theta_i)\cos(\phi)$ with $\cos(\theta_i - \phi) - \sin(\theta_i)\sin(\phi)$. Plugging that in gives Equation (5) right away. By Theorem 3.2 of [1], the polar part $1 - \cos(\theta_i - \phi)$ has a single minimum on $[0, \pi]$. Binary search finds the index $i^* = \arg\min_i |\theta_i - \phi|$ in $\lceil \log_2 N \rceil$ steps, and then we just pick the best fiber match $j^* = \arg\max_{j \in [i^* - R_{\max}, i^* + R_{\max}]} \langle w_j, \hat{z} \rangle$ inside the window. □

2.2 Surprisal Window, Self-Stopping, and Neighbor Recruitment

Once block j^* wins inside the local window, it produces scaled cosine logits $\ell_{j^*}(x) = s \cdot (W_{j^*}^{\text{exp}})^T \hat{z} + b_{j^*}^{\text{exp}} \in \mathbb{R}^C$ (with scale $s = 16.0$). From the cross-entropy loss $\mathcal{L}_{\text{CE}}(\ell_{j^*}(x), y)$, we get the Shannon surprisal in bits and the window radius:

$$\mathcal{I}(x, y; W_{j^*}) = \frac{\mathcal{L}_{\text{CE}}(\ell_{j^*}(x), y)}{\ln 2}, \quad r(x, y) = \min\left(R_{\max}, \left\lfloor \frac{\mathcal{I}(x, y; W_{j^*})}{2\kappa} \right\rfloor\right). \tag{6}$$

During training, two things happen depending on $r(x, y)$:

1. **Self-Stopping ($r = 0$):** If $r(x, y) = 0$ and $\mathcal{L}_{\text{CE}} \leq \epsilon_{\text{stop}}$, the model already knows this sample. We mask it out before backpropagation so it uses zero backward FLOPs.

2. **Recruiting a Neighbor ($r > 0$):** If the sample still has high error, we check if there is already a plastic block ($M_j < 1$) for class y in the local window. If the blocks sitting there are already locked on an old task ($M_j = 1$) or belong to other classes, SONT grabs a fresh, unused block ($M_{j_{\text{new}}} = 0$) in that sector, sets its key to the current input $w_{j_{\text{new}}} \leftarrow \hat{z}$, and re-sorts the sector angles so Equation (1) still holds.

3 Mathematical Framework for Scaling SONT

Now we can prove the four results needed to scale this setup to large dimensions and deep networks.

3.1 Pillar 1: Zero Drift with a Frozen Backbone and Maturity Gate

When we used raw-pixel PCA on Fashion-MNIST, the projection matrix P was built only from Task 1 images. Later tasks got squashed into that same subspace. Even worse, if an old block from Task 1 is allowed to take small gradient steps when Task 3 or Task 5 comes along, its weights slowly get overwritten.

Theorem 3.1 (Zero Consolidated Drift Under Maturity Attenuation)**.** *Let $\Phi_{frozen} : \mathcal{X} \to \mathbb{R}^D$ be a frozen feature backbone followed by a fixed Stiefel projection $P \in \mathrm{St}(D, d)$ and mean $\mu \in \mathbb{R}^D$. Suppose every expert block $j \in \{1, \ldots, N\}$ multiplies its weight update and Riemannian key update [6] by the maturity gate:*

$$\psi(M_j) = (1 - M_j)^\beta, \quad \beta \geq 1, \quad M_j \in [0, 1], \tag{7}$$

so that for learning rates $\eta_{exp}, \eta_{key} > 0$ and logit gradient $g_\ell = \nabla_\ell \mathcal{L}_{CE}(\ell_j(x_t), y_t)$:

$$\Delta W_j^{exp}(t) = -\eta_{exp}\psi(M_j)\,\hat{z}(x_t)g_\ell^T, \quad g_{tan,j}(t) = \psi(M_j)\big(\hat{z}(x_t) - \langle w_j, \hat{z}(x_t)\rangle w_j\big). \tag{8}$$

Then once a block j^* finishes training on Task $\mathcal{T}_a$ and locks ($M_{j^*} = 1$), the spherical features $\hat{z}(x)$ ($x \in \mathcal{T}_a$), the expert weights $(W_{j^*}^{exp}, b_{j^*}^{exp})$, and the routing key w_{j^*} have strictly zero drift during all later tasks $\mathcal{T}_b$ ($b > a$):

$$\big\|\Delta\hat{z}(x)\big\|_2 = 0, \quad \big\|\Delta W_{j^*}^{exp}(t)\big\|_F = 0, \quad \big\|\Delta w_{j^*}(t)\big\|_2 = 0 \quad \forall (x_t, y_t) \in \mathcal{T}_b. \tag{9}$$

Proof. Because Φ_{frozen}, μ, and P are fixed, the projection $\hat{z}(x) = \frac{(\Phi_{\text{frozen}}(x)-\mu)P}{\|(\Phi_{\text{frozen}}(x)-\mu)P\|_2}$ never changes over time, so $\|\Delta\hat{z}(x)\|_2 = 0$.

Now take any block j^* that was locked at the end of Task $\mathcal{T}_a$, meaning $M_{j^*} = 1$, and take any sample $(x_t, y_t) \in \mathcal{T}_b$ from a later task $b > a$. If the sample self-stops ($r = 0$ and $\mathcal{L}_{\text{CE}} \leq \epsilon_{\text{stop}}$), no weights are updated at all. If the sample does need an update, the recruitment rule sends the update to a plastic block ($M_j < 1$) or a fresh slot ($M_j = 0$). Even if j^* were selected, plugging $M_{j^*} = 1$ into Equation (7) gives $\psi(M_{j^*}) = (1 - 1)^\beta = 0$. Putting $\psi(M_{j^*}) = 0$ into Equation (8) makes $\Delta W_{j^*}^{\text{exp}}(t) = 0$, $\Delta b_{j^*}^{\text{exp}}(t) = 0$, and $g_{\text{tan},j^*}(t) = 0$. So the key update $w_{j^*} \leftarrow \frac{w_{j^*}+0}{\|w_{j^*}+0\|_2} = w_{j^*}$ does not move w_{j^*} at all. Note that when a sector is re-sorted by angle θ, a block's slot index in the array can shift left or right, but tracking the block by its unique ID shows that its actual weight matrices and key vector change by strictly 0. □

3.2 Pillar 2: Multi-Pole Spherical k-D Trees for High Dimensions

Let us look at why single-pole sorting breaks down when d is large. If we pick a random unit vector $\hat{z} \in \mathcal{S}^{d-1}$ in high dimensions, its dot product with a fixed pole $u_0 \in \mathcal{S}^{d-1}$ has mean 0 and variance $1/d$. By Levy's concentration of measure on the sphere:

$$\mathbb{P}\big(\big|\langle\hat{z}, u_0\rangle\big| \geq \epsilon\big) \leq 2\exp\left(-\frac{d\epsilon^2}{2}\right) \implies \mathrm{Var}\big(\theta(\hat{z})\big) \approx \frac{1}{d}. \tag{10}$$

When $d = 4096$, the variance of the angle $\theta(\hat{z})$ is $1/4096 \approx 0.000244$. Almost every point on the sphere sits right on the equator $\theta \approx \pi/2$. If all N blocks have the same angle $\approx \pi/2$ from u_0, sorting them along u_0 does not separate them. To fix this without losing fast binary search, we use k mutually perpendicular poles (expanding on Remark 3.4 of [1]).

Definition 3.2 (Orthogonal Multi-Pole Spherical k-D Tree)**.** Let $U = [u_1, u_2, \ldots, u_k] \in \mathrm{St}(d, k)$ be k orthonormal poles ($\langle u_p, u_q\rangle = \delta_{pq}$, $1 \leq k \ll d$). For any unit vector $v \in \mathcal{S}^{d-1}$, we define its residual vectors step-by-step across levels $p \in \{1, \ldots, k\}$ starting at $v^{(1)} = v$:

$$\theta_p(v) = \arccos(\langle v^{(p)}, u_p\rangle) \in [0, \pi], \quad v^{(p+1)} = \frac{v^{(p)} - \langle v^{(p)}, u_p\rangle u_p}{\big\|v^{(p)} - \langle v^{(p)}, u_p\rangle u_p\big\|_2} \in \mathcal{S}^{d-p-1}. \tag{11}$$

A set of $N = B^k$ unit blocks $\mathcal{W} = \{W_1, \ldots, W_N\} \subset \mathcal{S}^{d-1}$ forms a Multi-Pole Spherical k-D Tree with branching factor B if level 1 splits $\mathcal{W}$ into B buckets sorted by $\theta_1(W_i)$, and each level-p bucket is split into B sub-buckets sorted by $\theta_p(W_i)$. In practice, a very fast setup for continual learning is to use $B = 2$ on the first $k-1$ poles—which splits $\mathcal{S}^{d-1}$ into 2^{k-1} octants/sectors using the signs $\text{sign}(\langle \hat{z}, u_p \rangle)$—and then sort the $N/2^{k-1}$ blocks inside each sector by their residual angle $\theta_k(W_i)$ along pole u_k.

Theorem 3.3 (Multi-Pole Unimodal Search in $\mathcal{O}(k \log_2 N)$ Steps)**.** *Under Definition 3.2, let $\hat{z} \in \mathcal{S}^{d-1}$ have multi-pole angles $\boldsymbol{\phi}(\hat{z}) = (\phi_1(\hat{z}), \ldots, \phi_k(\hat{z})) \in [0, \pi]^k$. At level $p \in \{1, \ldots, k\}$, the residual vector $v^{(p)}$ is simply v with its components along $u_1, \ldots, u_{p-1}$ stripped out and re-normalized:*

$$v^{(p)} = \frac{v - \sum_{q=1}^{p-1} \langle v, u_q \rangle u_q}{\left\| v - \sum_{q=1}^{p-1} \langle v, u_q \rangle u_q \right\|_2}. \tag{12}$$

At every level p, the alignment error $E_p(i; \hat{z}) = \mathcal{L}_0 - \alpha_p \cos(\theta_p(W_i) - \phi_p(\hat{z}))$ ($\alpha_p > 0$) has a single valley over the sorted sub-buckets of level p, and binary search down the tree finds the leaf block i^* in:

$$K_{steps} = k \lceil \log_2 B \rceil \leq \log_2 N + k = \mathcal{O}(k \log_2 N) \text{ scalar comparisons.} \tag{13}$$

Proof. First let us check Equation (12) by induction on p. At $p = 1$, the sum is empty and $v^{(1)} = v$. Assume Equation (12) holds at level p, so $v^{(p)} = c_p(v - \sum_{q=1}^{p-1} \langle v, u_q \rangle u_q)$ where $c_p = \|v - \sum_{q=1}^{p-1} \langle v, u_q \rangle u_q\|_2^{-1} > 0$. Because the poles are mutually orthogonal ($\langle u_q, u_p \rangle = 0$ for all $q < p$), taking the dot product with u_p gives $\langle v^{(p)}, u_p \rangle = c_p \langle v, u_p \rangle$. Subtracting $\langle v^{(p)}, u_p \rangle u_p$ from $v^{(p)}$ gives:

$$v^{(p)} - \langle v^{(p)}, u_p \rangle u_p = c_p \left(v - \sum_{q=1}^{p} \langle v, u_q \rangle u_q \right).$$

When we divide this vector by its norm to get $v^{(p+1)}$, the positive constant c_p cancels out top and bottom, giving Equation (12) for $p + 1$.

Next, at level p, both $W_i^{(p)}$ and $\hat{z}^{(p)}$ are unit vectors on $\mathcal{S}^{d-p}$, and their angles $\theta_p(W_i), \phi_p(\hat{z})$ with u_p sit in $[0, \pi]$. Taking the derivative of $E_p(\theta_p) = \mathcal{L}_0 - \alpha_p \cos(\theta_p - \phi_p(\hat{z}))$ with respect to θ_p gives:

$$\frac{\partial E_p}{\partial \theta_p} = \alpha_p \sin\big(\theta_p - \phi_p(\hat{z})\big).$$

Since both angles are in $[0, \pi]$, their difference is in $[-\pi, \pi]$. That means the derivative is negative when $\theta_p < \phi_p(\hat{z})$, zero at $\theta_p = \phi_p(\hat{z})$, and positive when $\theta_p > \phi_p(\hat{z})$. Because the B sub-buckets at level p are sorted by θ_p, E_p goes straight down and then straight up. Binary search finds the best sub-bucket at level p in $\lceil \log_2 B \rceil$ steps. Doing this across all k levels with $B = N^{1/k}$ takes $k \lceil \log_2(N^{1/k}) \rceil \leq \log_2 N + k$ comparisons in total. □

3.3 Pillar 3: Decoupled Forward-Backward Routing in Deep L-Layer Networks

When we stack L SONT layers on top of each other, hidden layers $\ell \in \{1, \ldots, L-1\}$ do not have the ground-truth label y during the forward pass. To make deep stacking work, we route the forward pass using only the angle of the incoming hidden state, and we size the backward update window using the norm of the gradient coming back down from the loss.

Theorem 3.4 (Decoupled Forward Routing and Backward Gradient Surprisal)**.** *Take an L-layer network where each layer $\ell \in \{1, \ldots, L\}$ has N unit-sphere blocks $\mathcal{W}^{(\ell)} = \{W_1^{(\ell)}, \ldots, W_N^{(\ell)}\} \subset \mathcal{S}^{d-1}$ sorted by angle $\theta(W_i^{(\ell)}) = \arccos(\langle W_i^{(\ell)}, u_0^{(\ell)} \rangle)$.*

1. ***Forward Pass (No Label y Needed):*** *Let $\hat{h}^{(\ell-1)} = \frac{h^{(\ell-1)}}{\|h^{(\ell-1)}\|_2} \in \mathcal{S}^{d-1}$ be the normalized hidden state coming into layer ℓ, with angle $\phi(\hat{h}^{(\ell-1)}) = \arccos(\langle \hat{h}^{(\ell-1)}, u_0^{(\ell)} \rangle) \in [0, \pi]$. The forward alignment error:*

$$E_{fwd}^{(\ell)}(i; \hat{h}^{(\ell-1)}) = 1 - \cos\big(\theta(W_i^{(\ell)}) - \phi(\hat{h}^{(\ell-1)})\big) \tag{14}$$

has a single valley over $\{1, \ldots, N\}$, so binary search finds $i^{*(\ell)} = \arg\min_i |\theta(W_i^{(\ell)}) - \phi(\hat{h}^{(\ell-1)})|$ in $\lceil \log_2 N \rceil$ steps without needing y.

2. ***Backward Pass (Gradient Surprisal Window):*** *On the backward pass, let $\delta^{(\ell)} = \nabla_{h^{(\ell)}} \mathcal{L}(x, y)$ be the loss gradient reaching layer ℓ. We define the layerwise backward surprisal in bits and the update radius as:*

$$\mathcal{I}_{bwd}^{(\ell)}(x, y) = \frac{1}{\ln 2} \|\delta^{(\ell)}\|_2 \cdot \|\hat{h}^{(\ell-1)}\|_2 = \frac{\|\delta^{(\ell)}\|_2}{\ln 2}, \quad r^{(\ell)}(x, y) = \min\left(R_{\max}, \left\lfloor \frac{\mathcal{I}_{bwd}^{(\ell)}(x, y)}{2\kappa_\ell} \right\rfloor\right). \tag{15}$$

As the network learns a pattern so that $P(y \mid x) \to 1$, the top-layer gradient shrinks to zero ($\|\delta^{(L)}\|_2 \to 0$). That forces $\|\delta^{(\ell)}\|_2 \to 0$ and $r^{(\ell)}(x, y) = 0$ at every hidden layer ℓ, shutting off hidden updates automatically. On the other hand, when an unlearned sample causes a big gradient $\|\delta^{(\ell)}\|_2 \geq 2\kappa_\ell \ln 2$, $r^{(\ell)}(x, y) \geq 1$ opens up the window $[i^{*(\ell)} - r^{(\ell)}, i^{*(\ell)} + r^{(\ell)}]$ to update neighbors at layer ℓ.

Proof. Part 1 is just Theorem 3.2 of [1] with $\mathcal{L}_0 = 1$ and $\alpha = 1$, since both $\theta(W_i^{(\ell)})$ and $\phi(\hat{h}^{(\ell-1)})$ sit in $[0, \pi]$.

For Part 2, because $\hat{h}^{(\ell-1)}$ is a unit vector ($\|\hat{h}^{(\ell-1)}\|_2 = 1$), the outer-product gradient at layer ℓ has Frobenius norm $\|\delta^{(\ell)}(\hat{h}^{(\ell-1)})^T\|_F = \|\delta^{(\ell)}\|_2 \|\hat{h}^{(\ell-1)}\|_2 = \|\delta^{(\ell)}\|_2$. At the output layer L under cross-entropy loss, the logit gradient is $p(x) - e_y$, which satisfies $\|\delta^{(L)}\|_2 = \|p(x) - e_y\|_2 \leq \sqrt{2}\big(1 - P(y \mid x)\big)$. By the chain rule through the layer Jacobians $J_m = \frac{\partial h^{(m+1)}}{\partial h^{(m)}}$:

$$\|\delta^{(\ell)}\|_2 \leq \left(\prod_{m=\ell}^{L-1} \|J_m\|_2\right) \|\delta^{(L)}\|_2 \leq \sqrt{2} \left(\prod_{m=\ell}^{L-1} \|J_m\|_2\right) \big(1 - P(y \mid x)\big).$$

When the network gets the sample right ($P(y \mid x) \to 1$), the right-hand side goes to 0. Once $\|\delta^{(\ell)}\|_2 < \min(2\kappa_\ell \ln 2, \epsilon_{\text{stop}})$, we get $r^{(\ell)}(x, y) = 0$ and the hidden key update stops. When a sample produces a large error that reaches layer ℓ with $\|\delta^{(\ell)}\|_2 \geq 2\kappa_\ell \ln 2$, Equation (15) gives $r^{(\ell)}(x, y) \geq 1$, opening the neighborhood slice $[i^{*(\ell)} - r^{(\ell)}, i^{*(\ell)} + r^{(\ell)}]$. □

3.4 Pillar 4: Contiguous-Slice GPU Execution vs. Scattered MoE

There is a simple hardware reason why standard Top-K MoE is slow on GPUs. To pick K experts out of N, Top-K MoE first has to multiply the batch of B tokens by a dense router matrix $W_{\text{router}} \in \mathbb{R}^{d \times N}$ ($2BdN$ FLOPs). Then it runs 'topk' over all N scores, and finally it gathers K expert weight blocks $\{i_1, \ldots, i_K\}$ from random, non-adjacent spots in GPU memory (HBM).

SONT skips both problems. There is no $d \times N$ router matrix at all, and the active window $[i^* - r, i^* + r]$ is one continuous slice of $K = 2r + 1$ blocks sitting right next to each other in memory.

Proposition 3.5 (GPU Memory Coalescing and Routing Cost)**.** *Suppose N weight blocks of size d are stored in order in GPU HBM at addresses $\text{Addr}(W_i) = \text{Addr}_0 + i \cdot d$, with cache-line burst width L_{cache} blocks and active window size $K = 2r + 1 \ll N$. For a batch of B tokens:*

1. ***Top-K MoE:*** *Running the linear router and gathering K experts takes $\mathcal{O}(B{\cdot}d{\cdot}N{+}B{\cdot}K{\cdot}d)$ FLOPs. When $N \gg KL_{cache}$, the K scattered expert indices $\{i_1, \ldots, i_K\}$ force $\mathbb{E}[\mathcal{M}_{MoE}] = \Theta(K)$ separate HBM memory seeks per token.*

2. ***SONT Contiguous Slice:*** *Finding the input angle, running $\lceil \log_2 N \rceil$ binary search, and reading the contiguous slice $[i^* - r, i^* + r]$ takes only:*

$$\mathcal{F}_{SONT} = \mathcal{O}\big(B \log_2 N + B \cdot (2r+1) \cdot d\big) \text{ operations,} \tag{16}$$

with no $\mathcal{O}(B \cdot d \cdot N)$ router multiply. And because the blocks $W_{i^*-r}, \ldots, W_{i^*+r}$ sit in one continuous memory range $[\mathrm{Addr}_0 + (i^* - r)d,\ \mathrm{Addr}_0 + (i^* + r + 1)d)$, a kernel that keeps the $4N$-byte angle list in fast on-chip SRAM needs at most:

$$\mathcal{M}_{SONT} = \left\lceil \frac{2r+1}{L_{cache}} \right\rceil + 1 \text{ coalesced HBM burst reads per token.} \tag{17}$$

Proof. For each normalized token $\hat{h}_b$, getting $\phi(\hat{h}_b) = \arccos(\hat{h}_{b,1})$ takes $\mathcal{O}(1)$ work, and binary search over $\theta_1 \le \cdots \le \theta_N$ takes $\lceil \log_2 N \rceil$ scalar steps ($B\lceil \log_2 N \rceil$ total steps instead of $2BdN$ router multiply-adds). Once i_b^* is found, the $2r{+}1$ neighbor blocks sit side-by-side across $(2r{+}1)d$ floats in HBM. A continuous block of length $2r+1$ crosses at most $\lceil (2r+1)/L_{\text{cache}} \rceil + 1$ cache lines. In Top-K MoE, K indices spread across $\{1, \ldots, N\}$ have an average gap of $\frac{N}{K+1} \gg L_{\text{cache}}$, which means almost every expert fetch hits a different cache line ($\Theta(K)$ separate seeks). □

Algorithm 1 Contiguous-Slice GPU Execution for Multi-Pole SONT

Require: Sorted angle table $\Theta \in \mathbb{R}^{k \times N}$ (in SRAM/cache), contiguous expert tensor $W \in \mathbb{R}^{N \times d}$ (in HBM), batch of normalized states $\hat{H} \in \mathbb{R}^{B \times d}$, capacity κ, max radius $R_{\max}$.
Ensure: Output activations $Y \in \mathbb{R}^{B \times d_{\text{out}}}$, sparse Riemannian weight updates on $\mathcal{S}^{d-1}$.

parallel for token index $b = 0, \ldots, B-1$ **across GPU threads do**
Get multi-pole angles $\phi_1(\hat{h}_b), \ldots, \phi_k(\hat{h}_b)$ from $\hat{h}_b = \hat{H}[b, :]$.
// **Stage 1: $\mathcal{O}(k \log_2 N)$ Binary Search (No $d \times N$ Router MatMul)**
Find best polar index i_b^* using binary search ('searchsorted') on Θ.
// **Stage 2: Coalesced Contiguous Slice Read**
Read contiguous slice $W[i_b^* - R_{\max} : i_b^* + R_{\max} + 1]$ and compute output $Y[b, :]$.
// **Stage 3: Backward Surprisal Window & Maturity-Gated Update**
Get gradient δ_b; compute $\mathcal{I}_b = \|\delta_b\|_2 / \ln 2$ and $r_b = \min(R_{\max}, \lfloor \mathcal{I}_b / (2\kappa) \rfloor)$.
if $r_b > 0$ **or** $\|\delta_b\|_2 > \epsilon_{\text{stop}}$ **then**
    Update contiguous slice $j \in [i_b^* - r_b, i_b^* + r_b]$ with maturity gate $\psi(M_j) = (1 - M_j)^\beta$:

$$g_{\tan,j} = \psi(M_j)\big(g_j - \langle g_j, W_j \rangle W_j\big), \quad W_j \leftarrow \frac{W_j - \eta g_{\tan,j}}{\|W_j - \eta g_{\tan,j}\|_2}.$$

    Re-sort updated sector angles to keep $\theta_1 \le \cdots \le \theta_N$.
end if
end parallel for

4 Empirical Verification in PyTorch (Tesla T4 Benchmarks)

I ran two sets of benchmarks in PyTorch (seed = 42) on an NVIDIA Tesla T4 GPU to test everything from small streams up to 200-class vision datasets:

1. **Small Raw-Input Benchmarks:** 5-Task Split Fashion-MNIST ($D = 784$, 10 classes) and a live 140-D clinical ECG5000 stream (98% Normal Sinus flood).

2. **Large-Scale Vision and High-Dimensional Suite:** Checking Lemma 2.1 and Theorem 3.3 at $d = 4096, N = 4096$, followed by native 224×224 ViT-B/16 ($D = 768$) benchmarks on **10-Task Split ImageNet-R** (30,000 images, 200 classes), **ImageNet-A** (7,500 natural adversarial images), **4-Layer Deep Decoupled SONT**, and **Tesla T4 GPU latency**.

In the continual learning runs, I compare SONT against four baselines: (1) **Dense (AdamW)** (2-layer Residual MLP with LayerNorm and GELU), (2) **Dense + EWC** (Elastic Weight Consolidation [7] using online Fisher matrices), (3) **Switch Top-8 MoE** ($N_e = 64$ experts, top $k = 8$ softmax routing with Switch load-balancing loss [4] weighted by 0.02), and (4) **Dense + Experience Replay** (which saves raw images from past tasks in a memory buffer).

4.1 Small-Scale Tests: 5-Task Split Fashion-MNIST and ECG5000

In the first test (Table 1), Fashion-MNIST ($D = 784$, 10 classes) is split into 5 binary tasks $\mathcal{T}_1 = \{0, 1\}, \ldots, \mathcal{T}_5 = \{8, 9\}$ (2,000 train and 400 test images per task, 4 epochs per task, batch size 64). At test time, models must classify all 10 classes without task-ID hints. Here, single-pole SONT uses $N = 512, d = 64, \kappa = 0.45$ bits, $R_{\max} = 16$ on raw-pixel SVD features.

Table 1: 5-Task Split Fashion-MNIST Continual Learning ($D = 784$, 10 Classes, Raw Pixels, No Task-IDs at Inference).

Model / Algorithm	**Task 1 Final Acc**	**All-10-Class Avg Acc**	**Replay Buffer**	**Total**
Dense (AdamW)	0.00%	$\sim$ 19.80%	None (0)	
Dense + EWC ($\lambda = 800$)	0.00%	$\sim$ 19.85%	None (0)	
Switch Top-8 MoE (Load-Bal.)	0.00%	$\sim$ 19.75%	None (0)	
SONT (1-Pole, Buffer=0)	**42.00%**	**53.60%**	**None (0)**	
Dense + Replay (Buffer = 300)*	$\sim$ 72.00%	74.15%	300 raw images	

*Experience Replay stores raw past images in memory, which is not allowed in buffer-free continual learning.

Table 1 shows that Dense, EWC, and Switch MoE take 60,825 M to 86,016 M FLOPs and still drop to 0.00% on Task 1. SONT finishes all 5 tasks in just 223.6 M FLOPs (270× to 400× less compute) and keeps 42.00% on Task 1 (53.60% across all 10 classes) with zero replay buffer. On the 140-D ECG5000 clinical stream (800 balanced calibration patients followed by 2,000 live patients where 98% are Normal Sinus and 2% are Arrhythmia), SONT ($N = 256, d = 32$) holds $\approx 96.7\%$ balanced accuracy on 998 held-out test patients without a buffer.

4.2 Checking the Math at $d = 4096, N = 4096$

Before running the big vision datasets, I tested Lemma 2.1 and the Multi-Pole k-D Tree (Theorem 3.3) at $d = 4096, N = 4096$ ($k = 4$ poles, branching factor $B = 8$, since $8^4 = 4096$):

- **Fiber Identity (Lemma 2.1):** Across 1,000 random unit vector pairs in $\mathcal{S}^{4095}$ using 64-bit floats, the largest difference between $\langle w_i, \hat{z} \rangle$ and $\cos(\theta_i - \phi) - \sin(\theta_i)\sin(\phi)(1 - \langle f_i, f_z \rangle)$ was $\mathbf{2.949 \times 10^{-16}}$.
- **Equator Crowding vs. Multi-Pole Search (Theorem 3.3):** At $d = 4096$, random unit vectors had a measured polar angle variance of **0.00025**, matching the theoretical $1/d = 0.000244$ value in Equation (10). Even though all 4,096 blocks sit right near $\pi/2$ on any single axis, the 4-pole tree found the exact right leaf block **100.00%** of the time in just **12 scalar comparisons** instead of 4,096 dot products.

4.3 10-Task Split ImageNet-R (200 Classes, Native 224×224 ViT-B/16)

To see how SONT handles a hard modern vision benchmark, I ran it on **ImageNet-R** (30,000 images across 200 classes of cartoons, graffiti, origami, paintings, and sketches, split into 23,918 train and 6,082 test images). Every image is passed through a frozen ImageNet-1K pre-trained ViT-B/16 at native 224×224 resolution ($D = 768$). The 200 classes are split into 10 sequential tasks of 20 classes each (5 epochs per task, batch size 128, no task-IDs at test time).

To test how much Multi-Pole routing helps in $d = 512$ dimensions, I ran both **1-Pole SONT** ($k = 1$, all $N = 2048$ blocks in one list) and **4-Pole SONT** ($k = 4$, where the first 3 poles split $\mathcal{S}^{511}$ into $2^3 = 8$ octants of 256 blocks each, sorted along pole u_4).

Table 2: 10-Task Split ImageNet-R Continual Learning (200 Classes, Native 224×224 ViT-B/16, $N = 2048, d = 512$, No Task-IDs at Inference).

Model / Algorithm	Task 1 Final	Tasks 1–5 Avg	200-Class Avg	Buffer	Trai
Dense (AdamW)	0.00%	0.84%	8.02%	None (0)	543
Dense + EWC ($\lambda = 500$)	0.00%	1.90%	9.88%	None (0)	706
Switch Top-8 MoE (64 Exp.)	0.61%	5.94%	7.39%	None (0)	188
SONT (1-Pole, Proposed)	7.35%	9.03%	12.71%	None (0)	62
Multi-Pole SONT (4-Pole)	**32.01%**	**20.39%**	**20.20%**	**None (0)**	**56**
Dense + Replay (Buf = 1000)*	5.05%	6.45%	14.88%	1000 raw imgs	793

What Table 2 Shows:

1. **Beating a 1,000-Image Replay Buffer With Zero Buffer:** After 10 tasks (200 classes), Dense (0.00%), EWC (0.00%), and Switch MoE (0.61%) wipe out Task 1 almost completely. Even Dense + Experience Replay—which saves 1,000 raw images from old tasks—falls to 5.05% on Task 1 and 14.88% across all 200 classes. With **zero buffer**, 4-Pole SONT keeps **32.01**% on Task 1 (6.3× higher than Replay), **20.39**% on Tasks 1–5, and **20.20**% across all 200 classes, while using **14× fewer FLOPs** (56,458.1 M vs. 793,965.8 M).

2. **1-Pole vs. 4-Pole at $d = 512$:** Look at the gap between 1-Pole SONT and 4-Pole SONT. With 1 pole at $d = 512$, blocks crowd near the equator and Task 1 accuracy sits at 7.35% (12.71% overall). Splitting the sphere with 4 orthogonal poles jumps Task 1 retention more than 4× to **32.01**% and raises 200-class accuracy to **20.20**%, while actually using fewer FLOPs (56,458.1 M vs. 62,010.2 M).

3. **Zero Drift Verified (Theorem 3.1):** Checking the Task 1 blocks by their unique IDs after Task 10 finished showed $\|\Delta W_{j^*}^{\text{exp}}\|_F = \mathbf{0.000000}$ and $\|\Delta w_{j^*}\|_2 = \mathbf{0.000000}$. Across the run, **47.39**% of training samples self-stopped ($r = 0$) and skipped the backward pass.

4.4 ImageNet-A Adversarial Stream (10,000-Step 98% Flood)

Next, I tested 4-Pole SONT ($N = 512, d = 256, k = 4$) against an online Dense network on **ImageNet-A** (7,500 natural adversarial images that fool standard ImageNet models). Both models were calibrated on 1,000 ImageNet-R images and 1,000 ImageNet-A images, and then fed a live stream of 10,000 images made of 98% majority traffic (ImageNet-R) and only 2% ImageNet-A adversarial examples.

On 2,000 held-out test images, the 98% flood pushed online Dense down to 66.90% adversarial recall (83.15% balanced accuracy). Multi-Pole SONT kept **75.50**% **adversarial recall**

and **85.60% balanced accuracy** because its surprisal gate **stopped updates on **94.67%** of the 10,000-image stream**, refusing to let the repetitive 98% traffic overwrite the rare-class weights.

4.5 4-Layer Deep Decoupled SONT and Tesla T4 GPU Speedup

Finally, Stage 5 of the benchmark tested Pillar 3 (Theorem 3.4) and Pillar 4 (Proposition 3.5):

1. **4-Layer Deep SONT on 200-Class ImageNet-R:** I stacked $L = 4$ hidden 'Vectorized-DeepDecoupledSONTLayer' blocks ($d = 512, N = 512, \kappa = 0.22, R_{\max} = 8$) and trained the whole network end-to-end with PyTorch autograd on all 200 classes of ImageNet-R for 8 epochs. The 4-layer model reached **60.67% test accuracy across the 200 classes**. Better yet, looking inside the hidden layers from Epoch 1 to Epoch 8 shows Theorem 3.4 working in real time:
   - **Epoch 1:** The average hidden-layer backward radius was $r^{(\ell)} = \mathbf{1.06}$, and only **20.8%** of samples self-stopped.
   - **Epoch 8:** As the network converged, the average backward radius dropped to $r^{(\ell)} = \mathbf{0.01}$, and **99.5% of hidden-layer key updates turned themselves off automatically**, while all 4 layers kept their angles strictly sorted ($\theta_1 \leq \cdots \leq \theta_N$).
2. **Tesla T4 GPU Wall-Clock Latency (Table 3):** I timed forward routing and execution for $N = 4096$ blocks in $d = 1024$ dimensions with batch size $B = 512$ and radius $r = 4$ ($K = 2r+1 = 9$ active blocks per token, averaged over 200 CUDA-synchronized runs after 25 warmup runs).

Table 3: NVIDIA Tesla T4 GPU Latency ($N = 4096, d = 1024$, Batch Size $B = 512$, Active Window $K = 2r + 1 = 9$).

Method	Routing Cost	Tesla T4 Time	Speedup v
1. Dense Full Matrix Multiply	$\mathcal{O}(B \cdot N \cdot d)$	1.4982 ms / batch	1.16
2. Switch Top-9 MoE (Router + Gather)	$\mathcal{O}(B \cdot N \cdot d + BKd)$	1.7338 ms / batch	1.00
3. Proposed SONT (Binary Search + Slice)	$\mathcal{O}(\mathbf{B \log_2 N + BKd})$	**0.2083** ms / batch	**8.32**

Look at Row 2 of Table 3: on a real Tesla T4 GPU, Switch Top-9 MoE (1.7338 ms) is actually *slower* than just running a full Dense matrix multiply (1.4982 ms). That is because MoE still has to run the $d \times N$ router matrix multiply ('H @ router$_W$')$before running 'topk' and gathering scattered weights$ $r, i^* + r]$, it finishes the batch in **0.2083** ms—which is **7.19× faster than Dense** and **8.32× faster than Switch Top-9 MoE**.

5 Conclusion

In this paper, I looked at what it takes to scale Spectrally Ordered Neighborhood Training (SONT) from a single small layer up to deep networks and 200-class vision benchmarks in PyTorch. Pairing a frozen feature backbone with a maturity gate $\psi(M_j) = (1 - M_j)^\beta$ (Theorem 3.1) stops finished blocks from drifting when new tasks arrive ($\|\Delta W_{j^*}\|_F = 0$). Splitting the unit sphere across k orthogonal poles in a Multi-Pole Spherical k-D Tree (Theorem 3.3) fixes the high-dimensional equator crowding problem while keeping search at $\mathcal{O}(k \log_2 N)$ steps. On 10-Task Split ImageNet-R (200 classes, no task-IDs), moving from 1 pole to 4 poles pushes Task 1 accuracy from 7.35% up to 32.01% and reaches 20.20% across all 200 classes with zero buffer—beating AdamW (0.00%), EWC (0.00%), Switch MoE (0.61%), and even a 1,000-image

Experience Replay buffer (5.05% Task 1, 14.88% average) with 14× fewer FLOPs. Finally, splitting label-free forward routing from backward gradient surprisal (Theorem 3.4) lets a 4-layer SONT network reach 60.67% on 200-class ImageNet-R while shutting off 99.5% of hidden updates by Epoch 8, and contiguous-slice execution on a Tesla T4 GPU runs 8.32× faster in wall-clock time than Switch Top-K MoE.

References

[1] T. Zahoor, *Spectrally Ordered Neighborhood Training (SONT): Routerless Sparse Training via Unit-Sphere Ordering and Surprisal-Based Neighborhoods*, Preprint (2026).

[2] N. Halko, P. G. Martinsson, and J. A. Tropp, *Finding structure with randomness: Probabilistic algorithms for constructing approximate matrix decompositions*, SIAM Rev. 53 (2011), no. 2, 217–288.

[3] N. Shazeer et al., *Outrageously large neural networks: The sparsely-gated mixture-of-experts layer*, Proc. ICLR (2017).

[4] W. Fedus, B. Zoph, and N. Shazeer, *Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity*, J. Mach. Learn. Res. 23 (2022), no. 120, 1–39.

[5] C. E. Shannon, *A mathematical theory of communication*, Bell Syst. Tech. J. 27 (1948), no. 3, 379–423.

[6] P.-A. Absil, R. Mahony, and R. Sepulchre, *Optimization Algorithms on Matrix Manifolds*, Princeton University Press (2008).

[7] J. Kirkpatrick et al., *Overcoming catastrophic forgetting in neural networks*, Proc. Natl. Acad. Sci. U.S.A. 114 (2017), no. 13, 3521–3526.